CLAT-VSS v2

A hash-committed, masked threshold key vault over a 24-round Mersenne-31 permutation, and the cryptanalysis of its first version

Cryptographer Track submission
Mathematics for Systems Security hackathon

October 2026

Abstract

CLAT-VSS is a threshold key vault. A master key (MK) is wrapped under a key-encryption key (KEK), and the KEK is split T-of-N across custodians with verifiable secret sharing. Our first version (v1) combined a small LWE wrapper, Pedersen commitments, a lattice-style check, CRT residues and a 6-round SPN. Before resubmitting, we attacked it ourselves and found nine weaknesses: eight breaks in the code and a set of errors in the v1 paper. The master key leaks from public data. The commitment group falls to a 2^{16}-step discrete-log computation. The share checks are linear and therefore forgeable. The SPN has algebraic degree $5^6 < q$. The masks are constant across runs.

Version 2 removes every one of these components and rebuilds the vault on one public object: *CLAT-P*, a 24-round, full-round substitution-permutation network on $\mathbb{F}_q^{16}$ with $q = 2^{31} - 1$, the S-box x^5, and a 16×16 Cauchy MDS matrix. A sponge over CLAT-P provides hashing, commitments, Fiat-Shamir challenges and a deterministic authenticated key wrap (CLAT-SIV). The VSS uses only hash commitments, so it needs no discrete-log assumption, and its soundness error is $q^{-5} \approx 2^{-155}$ per challenge query. The combiner runs end to end on 2-share arithmetic masking with ISW multiplication and OS randomness.

The single-file C11 implementation (2792 lines, no dependencies beyond the C standard library, with `libm` for test statistics, and the OS random source) passes 38 of 38 functional checks and 7 of 7 statistical tests on Windows, Linux and macOS (arm64), and runs with no report under AddressSanitizer and UndefinedBehaviorSanitizer. These include the dudect timing test (max $|t| = 1.71$ for CLAT-P and 0.49 for the masked unwrap), a simulated-leakage TVLA in value and transition models (0 leaking samples on the masked paths, 7170 on the unmasked control), and degree-growth tests that reproduce the integral property on reduced rounds and show why it fails at full rounds. A separate red-team harness ran five attacks and one hiding check against v2; no attack succeeded. Every experimental number in this paper is produced by the shipped code; the v1 breaks and the security bounds are derived analytically.

Source code, CI workflow and run logs: https://github.com/zouhirmahjoubi/clat-vss

1 Introduction

A key vault stores one long-lived secret and must survive three kinds of adversary:

- a mathematical adversary who reads every public value;
- an insider who controls some custodians, or the dealer;
- a physical adversary who observes timing or power on the device that recombines the shares.

The hackathon rules ask for an original construction that survives cryptanalysis, has a small attack surface, and does not use pre-existing algorithms to encrypt plaintext.

The v1 submission met the letter of these goals but not their substance. Section 3 breaks the code in eight ways, each derived from the published source, and lists the errors in its paper. We kept the parts that were sound: the Mersenne-31 field, the x^5 S-box and the Cauchy MDS idea. We discarded everything whose security rested on a small or linear structure.

Contributions.

1. A cryptanalysis of our own v1 (Section 3): master-key recovery from public data, a discrete-log break of the commitment group, forgeable share checks, an integral distinguisher and an interpolation attack on the 6-round SPN, and constant masks.

2. CLAT-P, a 24-round full-round permutation of $\mathbb{F}_q^{16}$, with a round-count argument that covers univariate and multivariate degree, differential and linear trails, and a comparison with Poseidon2 over the same field (Section 5.1).
3. A hash-committed VSS with Fiat-Shamir challenges and masking polynomials, with proofs of soundness ($Qq^{-5} + Q^2q^{-9}$) and perfect hiding up to the commitment term (Section 5.3).
4. A masked combiner: commitment hashing, consistency checks, Lagrange interpolation and SIV unwrap all run on two arithmetic shares, and the MK leaves the device still split (Section 5.4).
5. A test suite that measures each claim, and a red-team harness that attacks v2 directly (Sections 7 and 8).

2 Threat model and goals

We map the goals to the ITU-T X.800 security services covered in both reference courses [15, 18].

- **Confidentiality of the MK.** An adversary who sees the wrapped blob, the VSS transcript, the header, and the shares of up to $T-1$ custodians learns nothing about the KEK from the sharing (Theorem 3), and against the MK gains at most the CLAT-SIV advantage of Section 5.2.
- **Integrity and data-origin authentication.** A modified blob, header, nonce or tag is rejected and the output is wiped. A custodian who sends a wrong share is detected and named.
- **Verifiability against a cheating dealer.** If all honest custodians accept, their shares lie on one polynomial of degree at most $T-1$, so any T of them reconstruct the same KEK.
- **Availability.** Any T honest custodians suffice.
- **Implementation security.** Code on secret data runs in constant time (no secret-dependent branches, memory indices, `%` or `/`). The combiner is first-order secure in the probing model, and no secret survives on the stack after a call returns.

Assumptions. The transcript is published over an authenticated broadcast channel, so all custodians see the same (T, N, c_i, g_k). CLAT-P is modelled as a random permutation in the sponge arguments, and the hash as a random oracle in the VSS proofs. The physical adversary is first-order: one probe, or one leakage sample, per observation. Higher-order attacks are out of scope (Section 10).

3 Cryptanalysis of v1

The v1 code is shipped unchanged in `v1/clat_vss_v1.c`. Every break below follows from that code and its constants. Table 1 summarises them, and Section 9 maps the course concepts behind most of them.

3.1 B1: the master key leaks from public data

The v1 ciphertext carries an LWE-style component per coordinate:

$$v_i^{\mathrm{hi}} = b_i r_i + e_i + \Delta\, m_i^{\mathrm{hi}}, \qquad v_i^{\mathrm{lo}} = b_i r_i + e_i' + \Delta\, m_i^{\mathrm{lo}} \pmod q,$$

with $r_i, e_i, e_i' \in [-4, 4]$ and $\Delta = 2^{15}$. The product $b_i r_i$ is a Hadamard product, not an inner product. For each coordinate there are only nine candidates for r_i. For the right one, $v_i - b_i r_i$ lies within ± 4 of a multiple of Δ, and that multiple is the message limb. Both halves share the same r_i, so the candidate is almost always unique.

Even without b, $v_i^{\mathrm{hi}} - v_i^{\mathrm{lo}} = \Delta(m_i^{\mathrm{hi}} - m_i^{\mathrm{lo}}) + (e_i - e_i')$ exposes the message difference. Decryption does not use this component to recover the message (it enters only the authentication tag), so it adds leakage and no security. The LWE dimension is $n = 8$ with secrets in $[-4, 4]^8$, a space of $9^8 \approx 2^{25.4}$, and the scheme has no module structure despite its "Module-LWE" label.

3.2 B2: the Pedersen group is too small

Commitments live in the subgroup of order $q = 2^{31} - 1$ of $\mathbb{F}_P^*$, $P = 46q + 1 \approx 2^{36.5}$. Generic algorithms (baby-step giant-step [25], Pollard rho [21]) compute $x = \log_g h$ in about $\sqrt{q} \approx 2^{15.5}$ group operations. Knowing x, the commitment is no longer binding. A change δy to a share word is compensated by changing its blinding word by $\delta z = -\delta y / x$, which leaves the commitment unchanged.

3.3 B3 and B4: linear checks are not checks

The "Ajtai/SIS" check multiplies a share by a public matrix. Shares are uniform in $\mathbb{F}_q$, so no shortness holds, and each half gives 4 linear equations in 16 unknowns. Any δy in the kernel passes. Combined with B2, substituting $\delta z = -\delta y/x$ leaves 4 Ajtai equations in the 8 unknowns δy of each half, so a nonzero forged share satisfies the Pedersen, Ajtai and CRT checks at once.

The CRT residues are recomputed from y by whoever edits y, which is exactly what the v1 tamper test does. They detect random faults only. This is the Hill-cipher lesson from both courses: a linear relation is solvable by linear algebra.

3.4 B5 and B9: constant masks and constant seeds

The masking seed is a compile-time string, so the masks are identical in every run. The state is also unmasked before every inverse S-box, and reconstruction loads shares in the clear. All nonces and seeds are constant strings, so every v1 random value, the masks included, is a fixed function of public constants. The CSPRNG also adds the output of a 128-bit LFSR; Berlekamp-Massey [14] recovers such a linear generator from 256 bits of its own output, so it adds no strength beyond the sponge it is mixed into.

3.5 B6: variable-time reduction

`fp_mul` reduces a 128-bit product with `%`, which compiles to a call to `__umodti3`. Its running time depends on the operands, and it runs on secret exponents [13].

3.6 B7: low algebraic degree of the 6-round SPN

Each v1 round applies $x \mapsto x^5$, so after R rounds every output word is a polynomial of degree at most 5^R in any single input word. For $R = 6$ this is $15625 < q - 1$.

Lemma 1 (integral property)**.** *Let $H \leq \mathbb{F}_q^*$ be the subgroup of order d, $d \mid q-1$, and let E be any map whose restriction to the line $t \mapsto a + te_0$, with e_0 the first unit vector, is a polynomial of degree $D < d$. Then $\sum_{t \in H} E(a + te_0) = d \cdot E(a)$.*

Proof. Write $E(a + te_0) = \sum_{j=0}^{D} \gamma_j t^j$. For $0 < j < d$, $\sum_{t \in H} t^j = 0$, because $t \mapsto t^j$ is a nontrivial character of the cyclic group H. Only $j = 0$ survives, which contributes $d\gamma_0 = d \cdot E(a)$. □

With $q - 1 = 2 \cdot 3^2 \cdot 7 \cdot 11 \cdot 31 \cdot 151 \cdot 331$ and $d = 32767 = 7 \cdot 31 \cdot 151 > 5^6$, Lemma 1 gives a probability-1 distinguisher for the 6-round v1 SPN, viewed as a keyed permutation, from $d + 1$ chosen plaintexts (the d coset points and a) [12]. The v1 wrap feeds the SPN only 16-bit words, which cannot contain a coset of H, so through the v1 interface the interpolation attack [11] is the operative one: $5^6 + 1 = 15626$ chosen plaintexts on one line fit within the 2^{16} values of a word and determine the encryption polynomial on that line, so every further ciphertext on the line can be predicted without the key.

3.7 B8: errors in the v1 paper

- The v1 paper gives $\Delta = \lfloor q/2^{16} \rfloor = 32768$; the floor is 32767. The code uses $\Delta = 2^{15} = 32768$, the value used in B1.
- P is described as a "safe subgroup modulus", but it is not a safe prime.
- A measured 272.86 μs timing window is a property of one machine, not a security property.
- The timing test compares two fixed plaintexts instead of fixed against random inputs.
- The SAC, Golomb and entropy statistics are sanity checks, not cryptanalysis.

4 Design of v2

In v1, CLAT stood for CRT-Lattice Algebraic Threshold. In v2 it stands for *Cauchy-Lagrange Algebraic Threshold*: a Cauchy MDS layer inside the permutation, and Lagrange interpolation in the threshold layer.

Break	Cause	v2 response
B1 key leak	LWE part with $n = 8$, small r shared by both halves	no LWE; the MK is wrapped with CLAT-SIV (Section 4.4)
B2 DLP	31-bit subgroup, $\approx 2^{16}$ generic steps	no group; hash commitments
B3 SIS check	linear map on uniform shares	commitment binding plus a random-linear-combination check
B4 CRT check	residues recomputed by the editor	removed
B5 masks	compile-time seed; unmasked before S^{-1}	OS randomness, wiped per call; masked end to end
B6 timing	128-bit `%`	Mersenne folding; no `%` or `/` on secrets
B7 degree	$5^6 < q$	$5^{24} > q - 1$, 8-round margin (Section 5.1)
B8 paper	wrong constants, statistics presented as proof	every number measured by the code
B9 seeds	constant nonces; LFSR in the CSPRNG	OS CSPRNG only

Table 1: The nine v1 breaks and how v2 closes each.

4.1 Field

All arithmetic is in $\mathbb{F}_q$, $q = 2^{31} - 1$. A product $x < 2^{62}$ is reduced by two folds $x \leftarrow (x \bmod 2^{31}) + \lfloor x/2^{31} \rfloor$ (shifts and masks, since $2^{31} \equiv 1$) and one masked conditional subtraction. Inversion is a^{q-2} by a Montgomery ladder with a fixed number of iterations.

The self-test stage of the suite (Test 2) checks three facts:

- 7 is a primitive root: $7^{(q-1)/p} \neq 1$ for every prime $p \mid q - 1$;
- the factorisation of $q - 1$ multiplies back;
- $5 \cdot D_{\text{inv}} \equiv 1 \pmod{q-1}$ with $D_{\text{inv}} = 1717986917$.

The last check shows that x^5 is a permutation.

4.2 The permutation CLAT-P

The state is $x \in \mathbb{F}_q^{16}$, and we write P for CLAT-P. Round $r \in \{0, \ldots, 23\}$ is

$$x_i \leftarrow x_i + c_{r,i}, \qquad x_i \leftarrow x_i^5, \qquad x \leftarrow Mx,$$

with

$$c_{r,i} = 7^{16r+i+1} \bmod q, \qquad M_{ij} = (i + j + 16)^{-1} \bmod q.$$

M is the Cauchy matrix $M_{ij} = (\alpha_i + \beta_j)^{-1}$ with $\alpha_i = i$ and $\beta_j = 16 + j$ for $i, j \in \{0, \ldots, 15\}$; the α_i are distinct, the β_j are distinct, and every $\alpha_i + \beta_j \in [16, 46]$ is nonzero. Every square submatrix of a Cauchy matrix is nonsingular, so M is MDS with branch number $16 + 1 = 17$ [26]. The self-test checks all 256 1×1 and all 14400 2×2 minors, and 10^4 random $k \times k$ minors for $k \leq 16$. Every one is nonsingular.

The constants are consecutive powers of a primitive root, so they are nothing-up-my-sleeve values. All 24 rounds are full rounds: every word passes through the S-box in every round.

4.3 Sponge and domain separation

The sponge has rate 7 words (words 0 to 6) and capacity 9 words (words 7 to 15, 279 bits). The unkeyed hash $\mathsf{H}(\tau; u_0, \ldots, u_{n-1})$ works as follows:

1. set the rate to $(\tau, n, 0, \ldots)$ and the capacity to zero;
2. permute;
3. absorb u in 7-word blocks by field addition, zero-padding the last block (the length is fixed by the IV), and permute after each block;
4. squeeze the output, permuting between 7-word blocks.

Tags separate uses: $\tau = 1$ for commitments, 2 for challenges and 3 for known-answer tests.

Keyed modes place a 256-bit key, as 9 limbs $k_0, \ldots, k_8$ of 29 bits (k_8 holds 24 bits), in the capacity. Word 15 holds $k_8 + \text{DOM} \cdot 2^{29}$ with $\text{DOM} = 0, 1, 2$ (written DOM_{HASH}, DOM_{MAC}, DOM_{ENC}). This is injective only while $k_8 < 2^{29}$, so every unmasked entry point that takes a key (wrap, unwrap, deal) rejects limbs outside the key range in constant time. The masked unwrap receives its key by interpolating a sharing whose secret the dealer range-checked.

4.4 CLAT-SIV deterministic key wrap

We follow the SIV paradigm of Rogaway and Shrimpton [23]:

$$\begin{aligned} \text{tag} &= \mathsf{MAC}_K(\nu, A, m) && \text{(rate words 0..6 after absorbing } \nu\|A\|m\text{, 217 bits)} \\ C &= m + \mathsf{KS}_K(\text{tag}) && \text{(keystream: rate after each permutation, keyed with DOM}_{\text{ENC}}\text{)} \end{aligned}$$

Here K is the KEK, m is the MK as 9 limbs, ν is a 7-word nonce from the OS, A is the header (version, key id, T, N), and the MAC IV encodes the three lengths.

Unwrapping computes $m' = C - \mathsf{KS}_K(\text{tag})$, recomputes the tag and compares it in constant time. On failure the output is wiped. Because the keystream depends on the whole message through the tag, a nonce repeat reveals only whether two wrapped keys are equal. This misuse resistance is the reason to use SIV for key wrapping.

4.5 Hash-committed VSS

The secret is the KEK as 9 limbs $s_0, \ldots, s_8$.

Dealing. The dealer samples:

- $f_l(X) = s_l + \sum_{t=1}^{T-1} a_{l,t} X^t$ with $a_{l,t} \in \mathbb{F}_q$ uniform, for each limb l;
- five masking polynomials $r_k(X)$, $k = 0..4$, of degree at most $T-1$ with all coefficients uniform.

Custodian $i \in \{1, \ldots, N\}$ receives

$$\sigma_i = \big(f_0(i), \ldots, f_8(i),\ r_0(i), \ldots, r_4(i),\ \text{salt}_i\big), \qquad \text{salt}_i \in \mathbb{F}_q^9 \text{ uniform.}$$

The dealer then publishes:

$$\begin{aligned} c_i &= \mathsf{H}\big(1;\ i, f(i), r(i), \text{salt}_i\big) \in \mathbb{F}_q^9, \\ (d_{k,l}) &= \mathsf{H}\big(2;\ T, N, c_1, \ldots, c_N\big) \in \mathbb{F}_q^{45}, \\ g_k(X) &= r_k(X) + \textstyle\sum_l d_{k,l} f_l(X). \end{aligned}$$

Each g_k is published as its T coefficients, so $\deg g_k \le T-1$.

Verification. Custodian i (or the combiner, on masked data) accepts σ_i if and only if:

- the commitment opens, that is, c_i equals the hash of σ_i; and
- $g_k(i) = r_k(i) + \sum_l d_{k,l} f_l(i)$ for every k.

Both verifiers, the reference one and the masked one used by the combiner, first reject a malformed transcript: parameters out of range, $N > 16$, non-canonical words, or T and N different from the authenticated header. They also bind each share to the slot it arrived on, so a custodian cannot shift blame onto another.

Reconstruction. The combiner computes Lagrange coefficients at 0 for the custodian indices, which must be distinct values in $1..N$. The KEK is $\sum_i \lambda_i f(i)$, computed on masked data.

5 Security analysis

5.1 Round count of CLAT-P

Univariate degree. After R rounds the degree in one input word is at most 5^R, and Lemma 1 needs a subgroup of order $d > 5^R$. Proper subgroups ($d \le (q-1)/2$) run out at $R = 13$. Summing over the whole line ($H = \mathbb{F}_q^*$ plus $t = 0$) passes the first S-box layer unchanged, because $(x+c)^5$ permutes $\mathbb{F}_q$, so it only needs $5^{R-1} < q - 1$: it holds up to $R = 14$ and is unavailable from $R = 15$ on.

The suite reproduces Lemma 1 for $R = 1..7$, with a negative control for each (Table 3). The interpolation attack needs about $5^{R-1} + 1$ points on a line, which exceeds the field size at the same point.

Multivariate degree. Using k free input words, a zero-sum over the full space $\mathbb{F}_q^k$ is guaranteed when the total degree is below $k(q-1)$, at a cost of q^k evaluations; within a 2^{128} budget, $k \le 4$. The first S-box layer maps the box $\mathbb{F}_q^k \times \{\text{const}\}$ onto itself, so only the remaining $R-1$ rounds count and the condition is $5^{R-1} < k(q-1)$. Since

$$5^{14} = 6.10 \times 10^9 \;<\; 3(q-1) = 6.44 \times 10^9, \qquad 5^{15} = 3.05 \times 10^{10} \;>\; 4(q-1),$$

a forward zero-sum reaches 15 rounds with $q^3 \approx 2^{93}$ evaluations and no further within the budget. Starting from a box in the middle, one backward round adds degree $D_{\text{inv}} \approx 2^{30.7} < 3(q-1)$, so 16 rounds admit a zero-sum partition. This degree argument leaves an 8-round margin at $R = 24$, where the forward condition alone would need $k \ge 5{,}551{,}116$ words; the program prints these bounds at run time. They are properties of the public permutation. In the keyed modes the attacker controls only the 7 rate words and cannot choose a middle box, so the margin there is larger.

Differential and linear trails. For $a \neq 0$, $(x+a)^5 - x^5 = b$ is a degree-4 equation in x, so the differential uniformity of x^5 is at most 4 [16] and each active S-box has differential probability at most $4/q \approx 2^{-29}$. The branch number 17 forces at least 17 active S-boxes in any two consecutive rounds, which bounds a two-round characteristic by 2^{-493}. The same counting with the Weil-type bound $4/\sqrt{q}$ on correlations gives 2^{-229} per two rounds.

Comparison with Poseidon2. Poseidon2 [8, 20], the successor of Poseidon [7], over 31-bit prime fields with width 16 and x^5 uses 8 full rounds and 14 partial rounds for 128-bit security: 142 S-box evaluations. CLAT-P uses 24 full rounds, or 384 S-box evaluations. A partial round has one active S-box where a full round has sixteen, so this round count exceeds Poseidon2's on every axis. Full rounds also remove the subspace trails that motivate partial-round analysis [9].

Our constants and MDS matrix differ from Poseidon2's. The argument above is a margin argument, not a proof, and it depends on no Poseidon2 constant.

5.2 Sponge and CLAT-SIV

With CLAT-P modelled as a random permutation, the unkeyed sponge is indifferentiable from a random oracle up to about Q^2/q^9, where Q is the number of permutation queries [4]. The capacity is $c = 279$ bits, so collision resistance is about 2^{139}.

The keyed sponge with a 256-bit key in the capacity has the generic bound of Andreeva et al. [2]. The advantage is dominated by $Q/2^{256}$ (key guessing) and by terms of order $Q^2/2^{279}$. The SIV construction is a deterministic AE when the MAC is a PRF and the encryption is IV-based and secure [23]. The 217-bit tag bounds blind forgery at 2^{-217} per attempt.

5.3 VSS soundness and hiding

Theorem 2 (soundness)**.** *Model* H *as a random oracle with outputs uniform in* $\mathbb{F}_q$*. Let* S *be the set of honest custodians,* $|S| \ge T+1$*, and suppose a dealer making* Q *oracle queries outputs a well-formed transcript and shares that every custodian in* S *accepts. Write* $y_{i,l}$ *and* $z_{i,k}$ *for the words custodian* i *received in the positions of* $f_l(i)$ *and* $r_k(i)$*; a cheating dealer need not have used polynomials. Then, except with probability at most* $Q\,q^{-5} + Q^2 q^{-9}$*, there are polynomials* $F_0, \dots, F_8$ *of degree at most* $T-1$ *with* $y_{i,l} = F_l(i)$ *for all* $i \in S$ *and all* l*.*

Proof. Binding. Except with probability $Q^2 q^{-9}$, no two oracle queries collide on a 9-word output, so each accepted c_i has exactly one opening queried before c_i was fixed. The vectors $\phi_l = (y_{i,l})_{i \in S}$ and $\psi_k = (z_{i,k})_{i \in S}$ are therefore fixed before the oracle returns d.

Linear algebra. Let $W \subset \mathbb{F}_q^S$ be the evaluations on S of polynomials of degree at most $T-1$. Then $\dim W = T < |S|$. Let $\pi : \mathbb{F}_q^S \to \mathbb{F}_q^S/W$ be the quotient map. Acceptance by all of S for index k means that $\psi_k + \sum_l d_{k,l}\phi_l$ agrees on S with g_k, so $\pi(\psi_k) + \sum_l d_{k,l}\pi(\phi_l) = 0$.

Suppose some $\pi(\phi_{l^*}) \neq 0$. Then the map $\delta \mapsto \sum_l \delta_l \pi(\phi_l)$ from $\mathbb{F}_q^9$ to $\mathbb{F}_q^S/W$ is linear and nonzero. Its preimage of $-\pi(\psi_k)$ is empty or a coset of a proper subspace, so it has probability at most $1/q$ under a uniform d_k. The five rows $d_0, \dots, d_4$ are independent oracle output, so all five equations hold with probability at most q^{-5}. A union bound over the dealer's Q challenge queries gives Qq^{-5}.

Hence, except with the stated probability, $\pi(\phi_l) = 0$ for every l, which means $\phi_l \in W$. □

Consequently any T accepting honest custodians reconstruct the same KEK. Grinding a transcript to pass with an inconsistent sharing costs about $q^5 \approx 2^{155}$ hash evaluations.

Theorem 3 (hiding)**.** *Let the dealer be honest and let $\mathcal{C}$ be a set of at most $T-1$ custodians. In the random-oracle model, the joint distribution of the transcript and the shares of $\mathcal{C}$ is independent of the secret s, except with probability $Q\,q^{-9}$ over an adversary making Q oracle queries.*

Proof. Unless the adversary queries the oracle on an honest custodian's exact commitment input, which requires guessing a uniform 9-word salt, the honest commitments c_i are uniform and independent of everything else. That probability is at most Qq^{-9}; condition on it not happening. The challenge d is a function of the commitments.

Pad $\mathcal{C}$ to exactly $T-1$ points. A polynomial of degree at most $T-1$ corresponds one-to-one to its values on $\mathcal{C} \cup \{0\}$. For each l, the values $(f_l(i))_{i\in\mathcal{C}}$ are uniform and independent of $s_l = f_l(0)$, which is Shamir's argument [24]. For each k, the adversary can already compute g_k on $\mathcal{C}$ from its own shares. The only new value is

$$g_k(0) = r_k(0) + \textstyle\sum_l d_{k,l} s_l.$$

Since $r_k(0)$ is uniform and appears nowhere else in the view, $g_k(0)$ is uniform and independent of s. The whole view is therefore independent of s. □

Remark 1. The VSS rests only on hash functions modelled as random oracles, so it avoids the discrete-log assumption that Feldman [5] and Pedersen [19] VSS need. The random-linear-combination check follows the pattern of recent hash-based non-interactive VSS [1]. Our contribution is to instantiate it over a single Mersenne-31 permutation and to run its verifier on masked data.

5.4 Masking

A secret x is held as (x_0, x_1) with $x = x_0 + x_1$. Linear operations, including the MDS layer, constant addition and Lagrange combination with public λ_i, act on each share separately. Multiplication uses the ISW gadget [10] with fresh ρ:

$$z_0 = a_0 b_0 + \rho, \qquad z_1 = a_1 b_1 + \big((a_0 b_1 - \rho) + a_1 b_0\big),$$

evaluated in that order. The S-box computes x^2, x^4 and x^5 with a refresh of one operand before each multiplication, so the two inputs of a gadget never share a mask. This is the standard composition argument for first-order probing security [10, 3].

Masks come from the OS CSPRNG through a buffer. Each word is cleared after use, and the whole buffer is wiped on entry to and exit from every operation that consumes secret randomness.

The combiner pipeline is masked end to end:

1. shares arrive split;
2. commitment hashing and the g-check run on shares, and only the public digest and the check difference are unmasked;
3. masked Lagrange interpolation produces the masked KEK;
4. the masked SIV unwrap leaves the MK as two shares.

The tag comparison unmasks $\rho_j(\mathrm{tag}'_j - \mathrm{tag}_j)$ with a fresh nonzero ρ_j for each word, which reveals only whether each word matches. In the simulated trace, the two shares of a value are separated by a fresh random precharge or by unrelated values, so a Hamming-distance transition does not combine them. The one exception is the tag-comparison word $\rho_j(\mathrm{tag}'_j - \mathrm{tag}_j)$, which is unmasked anyway.

6 Implementation and attack surface

The implementation is one C11 file, `clat_vss.c` (2792 lines including the test suite). It depends only on libc, `libm` for test statistics, and the OS random source (`getrandom`, `getentropy`, or `RtlGenRandom`). It builds with `-std=c11 -O3 -Wall -Wextra -Werror -pedantic` and no warnings under GCC 16.2 (MinGW-w64), GCC 13.3 and Clang 18.1 (Ubuntu 24.04), and Apple Clang 15 (macOS 14, arm64). Measures that shrink the attack surface:

- **One primitive.** Hashing, commitments, challenges, MAC and keystream all use CLAT-P. There are no tables indexed by secrets, no key schedule and no second cipher.
- **Constant time.** There are no branches on secrets. Selects use masks behind compiler value barriers. Reduction uses shifts and adds. `%` and `/` appear only on public values (test statistics, subgroup orders, loop bounds).

Platform	Compiler	OS RNG	Timer	dudect $\lvert t\rvert$	Functional	Statistical
Windows 10, x86-64	GCC 16.2.0	`RtlGenRandom`	`rdtsc`	1.71 / 0.49	38/38	7/7
Ubuntu 24.04, x86-64 (CI)	GCC 13.3.0	`getrandom`	`rdtsc`	0.91 / 1.51	38/38	7/7
Ubuntu 24.04, x86-64 (CI)	Clang 18.1.3	`getrandom`	`rdtsc`	2.01 / 0.66	38/38	7/7
macOS 14, arm64 (CI)	Apple Clang 15.0.0	`getentropy`	`clock_gettime`	1.12 / 1.85	38/38	7/7
Ubuntu 24.04, ASan + UBSan	GCC 13.3.0, `-O1`	`getrandom`	`rdtsc`	1.71 / 1.69	36/36*	7/7

Table 2: The suite on four configurations. The dudect column gives CLAT-P / masked unwrap. In every run the red-team harness reports 0 successful attacks. *The sanitizer build skips the two dead-stack probes, because ASan's redzones change the stack layout they measure; the run produced no sanitizer report.

- **Input validation before arithmetic.** The code validates every public input before computing with it:
  - transcript parameters and canonical words;
  - T and N equal to the authenticated header;
  - custodian indices in $1..N$, distinct, and equal to the sender's slot;
  - key limbs in key range (unmasked entry points; only the verdict is public);
  - blob words canonical.

  A review found a stack overflow from a transcript with $N = 17$, which is now rejected before hashing and covered by a regression test.
- **No residue.** Secret buffers are wiped with volatile stores and a compiler barrier, and a test shows this survives optimisation (a dead buffer keeps 0 bytes of its pattern; with plain `memset`, 256 bytes remain). Each entry point that handles keys runs its body in a non-inlined function and then burns 8 KiB of dead stack. After `clat_siv_wrap`, 0 of 2048 scanned bytes are nonzero, against 256 without the burn.
- **No heap on cryptographic paths.** Only the test harness allocates memory.

7 Evaluation

All results come from one run of `./clat_vss` (`results/v2_run_verify.txt`) on an Intel Core i5-6300U at 2.40 GHz under Windows 10, built with GCC 16.2.0 (WinLibs, UCRT) at `-O3`. The run took 84.6 s and exited with status 0. Table 2 repeats the suite on Linux and macOS.

Known answers. Three known answers are fixed: $\mathsf{P}(0, 1, \ldots, 15)$ begins `5f0d27c8 4a31df17`, $\mathsf{H}(3; \varnothing)$ begins `0fcb86a3 77e744d8`, and the SIV blob for a fixed key and nonce has a ciphertext beginning `47d31d57` and a tag beginning `67266edd`. An independent Python implementation written from the specification (`tools/ref_kat.py`) reproduces all three.

Cross-platform runs. The same source was built with the flags above and run by GitHub Actions on Ubuntu 24.04 (x86-64) with GCC and Clang, on macOS 14 (arm64) with Apple Clang, and on Ubuntu with GCC under AddressSanitizer and UndefinedBehaviorSanitizer (Table 2). Each run uses that platform's OS random source. The three known answers match on every platform. CI also diffs the Python reference output against the recorded values. The logs are in `results/ci/`.

Vault demonstration (3-of-5). All 18 vault checks pass.

- Verifying all five split shares on the masked path takes 2.27 ms.
- Masked Lagrange interpolation from custodians $\{2, 4, 5\}$ plus the masked unwrap takes 0.38 ms and returns the MK exactly.
- 10 of 10 three-custodian subsets recover the KEK; 0 of 10 two-custodian subsets do.
- A one-bit flip in custodian 4's share is rejected with failure mask `0x10`, which names custodian 4.
- A v1-style *consistent* forgery for custodian 2 satisfies every g-check but fails its commitment (mask `0x04`). Used anyway, it gives a wrong KEK, the unwrap fails and the output is wiped.

R	5^R	$d > 5^R$	relation	control $d \le 5^R$	relation
1	5	6	holds	3	fails
2	25	31	holds	22	fails
3	125	126	holds	99	fails
4	625	651	holds	558	fails
5	3125	3171	holds	3069	fails
6	15625	19026	holds	14949	fails
7	78125	84258	holds	76461	fails
24	5.96×10^{16}	none ($5^{24} > q-1$)	n/a		

Table 3: Lemma 1 on reduced-round CLAT-P. Each d divides $q-1$. In every tested case the relation holds for $d > 5^R$ and fails for the control $d \le 5^R$; at 24 rounds $5^R > q-1$, so no subgroup qualifies.

Target	samples/trace	traces	HW max $\lvert t\rvert$	HD max $\lvert t\rvert$	over 4.5 (single set)	in both sets
unmasked CLAT-P	8080	2×2000	111.98 / 111.88	118.77 / 125.97	7135 to 7309	7170 / 7070
masked CLAT-P	28448	2×10000	4.13 / 4.68	4.18 / 4.40	0 to 1	0 / 0
combine	170835	2×3000	4.88 / 4.49	4.89 / 4.38	0 to 3	0 / 0
verify	170836	2×3000	4.36 / 4.81	4.97 / 4.52	0 to 2	0 / 0

Table 4: Simulated-leakage TVLA. "combine" is masked Lagrange plus masked unwrap; "verify" is masked commitment hashing plus the g-check. In the two max $\lvert t\rvert$ columns, a/b is set 1 / set 2. The single-set column gives the range of exceedance counts over both sets and both models. The last column counts samples over 4.5 in both independent sets, as HW / HD.

- A dealer's off-polynomial share for custodian 3 is caught by the g-check even though its commitment matches (mask `0x08`).
- Changes to the ciphertext, tag, nonce or header are rejected.
- Malformed transcripts, replayed shares, policy mismatches, out-of-range indices and out-of-range keys are all rejected, by the reference verifier and the masked verifier alike.

Constant time (dudect). We use fixed-versus-random classes, random interleaving, 10^5 measurements per target, `rdtsc` with `lfence`, and Welch's t at six percentile crops [22]. For CLAT-P, max $\lvert t\rvert = 1.71$. For the masked SIV unwrap, max $\lvert t\rvert = 0.49$. Both are below the 4.5 threshold. In the unwrap test the fixed class always takes the success path (49863 of 49863 opened) and the random class always fails (0 of 50137), so the success and failure paths are also indistinguishable in time.

Simulated leakage (TVLA). Each value the traced code stores (S-box and gadget intermediates, folded MDS products and outputs, absorbed sponge words, the Lagrange and g-check accumulators, keystream and tag-comparison words) passes through a hook that records its Hamming weight (HW, value model) and its Hamming distance to the previous value (HD, transition model). We run the fixed-versus-random t-test per sample on two independent data sets [6]. A sample counts as leaking only if $\lvert t\rvert > 4.5$ in both sets.

Under the null hypothesis $\Pr[\lvert t\rvert > 4.5] \approx 6.8 \times 10^{-6}$, so 170,836 samples give about 1.2 chance exceedances per set. The isolated single-set hits in Table 4 match that rate. The unmasked control leaks at 7170 (HW) and 7070 (HD) samples, which shows the test can detect leakage. The masked targets show 0. The long mode (`-full`, run in CI on Ubuntu with GCC) uses more traces: 2×10^5 for masked CLAT-P, and $2 \times 2 \times 10^4$ for the unmasked control, combine and verify. With 10 times more traces, the unmasked control's largest $\lvert t\rvert$ grows from about 120 to 412.47, close to the $\sqrt{10}$ growth expected from a real leak, while the masked maximum stays at 4.15 (HD) and every masked target again has 0 confirmed samples. The same run passes dudect with 10^6 (CLAT-P, max $\lvert t\rvert = 1.36$) and 4×10^5 (masked unwrap, max $\lvert t\rvert = 1.04$) measurements and all 38 functional checks. This is a simulation of the trace order in a value and a transition model. It is not a measurement of real hardware (Section 10).

Sanity statistics (not proof).

- Strict avalanche over 200 trials $\times$ 496 input bits: mean flip rate 0.499966 ($z = -0.48$), largest per-cell $\lvert z\rvert = 4.81$, and 0 of 246016 cells above 5.

- Bit balance: max $|z| = 2.24$.
- Byte entropy: 7.99961 bits per byte, with $\chi^2_{255} = 277.3$ against a 0.9999 quantile of 347.7.

These statistics would catch a gross implementation error. They say nothing about cryptanalytic strength.

8 Red-team results

A separate harness, `attacks/redteam_v2.c`, includes the v2 source and attacks its internal functions directly. Results are in `results/redteam_v2_run.txt`. None of the five attacks (A1, A3 to A6) succeeded; A2 is a hiding check that cannot fail.

A1 *Port of the v1 LWE leak.* Over 180000 ciphertext words, the mean distance to the nearest multiple of 2^{15} is 8200.8 (uniform: 8192), and $\Pr[\text{distance} \le 4] = 2.83 \times 10^{-4}$ (uniform: 2.75×10^{-4}). No small-error structure remains.

A2 *KEK from the transcript (hiding check, not an attack).* For any candidate KEK s, $r_k(0) = g_k(0) - \sum_l d_{k,l} s_l$ is a valid masking constant, so the transcript alone rules out no KEK (Theorem 3). The harness illustrates this on 100000 random candidates; the count follows from the argument and is not a measurement.

A3 *Integral distinguisher.* The relation holds at $R = 2$, $d = 31$ and at $R = 3$, $d = 151$, and fails in the controls at $R = 3$, $d = 31$ and $R = 4$, $d = 151$. At $R = 24$ no subgroup qualifies.

A4 *SIV forgery.* 0 of 713 single-bit flips across nonce, ciphertext and tag were accepted, and 0 of 300000 random forgeries.

A5 *Mask and salt reuse.* Salts differ across deals. The same share masked twice gives different splits that unmask to the same value.

A6 *Consistent share forgery.* The forgery passes every g-check and fails the commitment.

Before the red team ran, three independent reviews (cryptanalysis, side channels, correctness) produced 19 findings. 18 were fixed; the code-level fixes (transcript checks, header binding, blame by slot, index and key ranges, stack burn) have regression checks in Tests 2 and 3. One was rejected: the claim that the shared KEK should be provably bound to the blob. A dealer can always wrap garbage, and a wrong KEK is caught by the SIV tag at unwrap time.

9 Course concept map

The design was checked against two courses: Paar's 27-lecture *Introduction to Cryptography* [18, 17] and Neso Academy's 83-video *Cryptography & Network Security* [15]. Table 5 lists the concepts that changed the design. The full lecture-by-lecture map is in `docs/COURSE_NOTES.md`.

10 Limitations

- **No external cryptanalysis.** CLAT-P is new. Its round count rests on the degree and trail arguments above and on the comparison with Poseidon2, not on years of public analysis. Gröbner-basis attacks are covered only through that comparison.
- **Leakage is simulated.** The TVLA models the trace order in HW and HD models. Real power or EM traces can leak through register reuse, glitches and coupling that the model does not capture. The masking is first-order only.
- **Timing platforms.** The suite passes on Windows, Linux and macOS (Table 2). The Linux and macOS timing tests ran on shared CI virtual machines, and the arm64 run uses `clock_gettime` instead of a cycle counter, so those dudect figures are coarser than the laptop run. BSD, which shares the `getentropy` path with macOS, was not tested separately.
- **Threads.** The mask buffer is process-wide and unlocked, so callers must not run the masked entry points from two threads at once.
- **Quantum dealer.** Shor's algorithm does not apply to the VSS, but Grover search lets a quantum dealer find a passing challenge in about $q^{5/2} \approx 2^{78}$ queries. Raising the number of masking polynomials from 5 to 9 would restore about 2^{139}.

Lecture topic	Lesson	Where it shows up
Historical ciphers; Hill cipher	linear schemes fall to linear algebra	B3 and B4 broken; v2 uses hash commitments
LFSRs, Berlekamp-Massey	linear generators are predictable	B9; v2 uses the OS CSPRNG only
DES and AES: confusion and diffusion, MDS	a nonlinear layer and an MDS layer with high branch number	x^5 plus a Cauchy MDS with branch number 17
Galois fields	inverses, primitive roots	Fermat inversion; constants $7^{16r+i+1}$
Modes of operation	IV reuse is fatal for CTR and OFB	SIV: deterministic, misuse resistant
Euler, Fermat, EEA	$\gcd(5, q-1) = 1$, inverses mod $q-1$	x^5 is a permutation; D_{inv} checked
RSA: fast exponentiation	square-and-multiply leaks through timing	Montgomery ladder; dudect
DH, generalised DLP, BSGS	generic DLP in a group of order n costs $\sqrt{n}$	B2 broken in about 2^{16} steps; v2 has no DLP
CRT	residues need an independent source	B4 is a no-op
Groups, cyclic subgroups	subgroups of order $d \mid q-1$	integral distinguisher (Lemma 1)
Hash functions, SHA-3	sponge, capacity, birthday bound	rate 7, capacity 279 bits
Hash-based PQC, Merkle signatures	hash-only security survives Shor	DLP-free VSS (classical ROM proofs; Grover lowers the q^{-5} soundness to about 2^{78} quantum queries)
MAC, HMAC	keyed hashing, constant-time compare	keyed sponge MAC; wipe on failure
Key establishment	key-encryption keys, key hierarchy	MK under KEK, KEK split T-of-N
PKI, man in the middle	authenticated public data	broadcast assumption; header binds T and N
PRNG, Golomb postulates	statistics are not security	sanity tests labelled as such

Table 5: Course concepts that shaped v2.

- **Broadcast.** The VSS assumes all custodians see the same transcript. The protocol does not include a complaint round.
- **Course transcripts.** YouTube rate-limited the transcript downloads (HTTP 429), so the concept map is built from the lecture titles and the companion textbook [17], not from transcripts.

11 Conclusion

Breaking our own first version taught us more than any test it had passed: six of its nine breaks (B1 to B4, B7, B9) came from a structure that was too small or linear. Version 2 uses one permutation with an 8-round margin over the integral bound of Section 5.1, a VSS whose security reduces to hashing, and a combiner that never holds the KEK or the MK unmasked. The code measures every claim it makes, and the red team found no break.

References

[1] S. Atapoor, K. Baghery, D. Cozzo, R. Pedersen. VSS from distributed ZK proofs and applications. ASIACRYPT 2023. ePrint 2023/992.

[2] E. Andreeva, J. Daemen, B. Mennink, G. Van Assche. Security of keyed sponge constructions using a modular proof approach. FSE 2015.

[3] G. Barthe, S. Belaïd, F. Dupressoir, P.-A. Fouque, B. Grégoire, P.-Y. Strub, R. Zucchini. Strong non-interference and type-directed higher-order masking. ACM CCS 2016.

[4] G. Bertoni, J. Daemen, M. Peeters, G. Van Assche. On the indifferentiability of the sponge construction. EUROCRYPT 2008.

[5] P. Feldman. A practical scheme for non-interactive verifiable secret sharing. FOCS 1987.

[6] G. Goodwill, B. Jun, J. Jaffe, P. Rohatgi. A testing methodology for side-channel resistance validation. NIST Non-Invasive Attack Testing Workshop, 2011.

[7] L. Grassi, D. Khovratovich, C. Rechberger, A. Roy, M. Schofnegger. Poseidon: a new hash function for zero-knowledge proof systems. USENIX Security 2021.

[8] L. Grassi, D. Khovratovich, M. Schofnegger. Poseidon2: a faster version of the Poseidon hash function. AFRICACRYPT 2023.

[9] L. Grassi, R. Lüftenegger, C. Rechberger, D. Rotaru, M. Schofnegger. On a generalization of substitution-permutation networks: the HADES design strategy. EUROCRYPT 2020.

[10] Y. Ishai, A. Sahai, D. Wagner. Private circuits: securing hardware against probing attacks. CRYPTO 2003.

[11] T. Jakobsen, L. R. Knudsen. The interpolation attack on block ciphers. FSE 1997.

[12] L. Knudsen, D. Wagner. Integral cryptanalysis. FSE 2002.

[13] P. Kocher. Timing attacks on implementations of Diffie-Hellman, RSA, DSS, and other systems. CRYPTO 1996.

[14] J. L. Massey. Shift-register synthesis and BCH decoding. IEEE Trans. Inf. Theory 15(1), 1969.

[15] Neso Academy. Cryptography & Network Security (83 videos). YouTube playlist PLBlnK6fEyqRgJU3EsOYDTW7m6SUmW6kII.

[16] K. Nyberg. Differentially uniform mappings for cryptography. EUROCRYPT 1993.

[17] C. Paar, J. Pelzl, T. Güneysu. Understanding Cryptography, 2nd ed. Springer, 2024.

[18] C. Paar. Introduction to Cryptography (27 lectures, Ruhr University Bochum). YouTube playlist PL3boZvi-wmN6r4HUGUpRSk5uhEcTNfjSS.

[19] T. P. Pedersen. Non-interactive and information-theoretic secure verifiable secret sharing. CRYPTO 1991.

[20] Plonky3 contributors. `p3-poseidon2`, `round_numbers.rs`: 31-bit fields, width 16, degree 5 $\Rightarrow$ 8 full and 14 partial rounds.

[21] J. M. Pollard. Monte Carlo methods for index computation (mod p). Mathematics of Computation 32, 1978.

[22] O. Reparaz, J. Balasch, I. Verbauwhede. Dude, is my code constant time? DATE 2017.

[23] P. Rogaway, T. Shrimpton. A provable-security treatment of the key-wrap problem. EUROCRYPT 2006.

[24] A. Shamir. How to share a secret. Communications of the ACM 22(11), 1979.

[25] D. Shanks. Class number, a theory of factorization, and genera. Proc. Symp. Pure Math. 20, 1971.

[26] A. M. Youssef, S. Mister, S. E. Tavares. On the design of linear transformations for substitution permutation encryption networks. SAC 1997.